Online Book Store Data Analysis

SQL Database Schema, Data Manipulation & Analytical Queries Project

PROJECT TOPIC	COURSE MENTOR	DATABASE SYSTEM
Online Book Store SQL Project	Sathish Dhawale Sir	PostgreSQL / Standard SQL

1. Database Schema & Data Loading (DDL & DML)

Is section mein Relational Database Table structures design aur CSV Data Import execution process standard syntax mein explain kiya gaya hai:

Table Creation: Books, Customers & Orders

```
DROP TABLE IF EXISTS Books; CREATE TABLE Books ( Book_ID SERIAL PRIMARY KEY, Title VARCHAR(100), Author VARCHAR(100), Genre VARCHAR(50), Published_Year INT, Price NUMERIC(10, 2), Stock INT ); DROP TABLE IF EXISTS Customers; CREATE TABLE Customers ( Customer_ID SERIAL PRIMARY KEY, Name VARCHAR(100), Email VARCHAR(100), Phone VARCHAR(15), City VARCHAR(50), Country VARCHAR(150) ); DROP TABLE IF EXISTS Orders; CREATE TABLE Orders ( Order_ID SERIAL PRIMARY KEY, Customer_ID INT REFERENCES Customers(Customer_ID), Book_ID INT REFERENCES Books(Book_ID), Order_Date DATE, Quantity INT, Total_Amount NUMERIC(10, 2) );
```

Data Import Script (CSV Files via PostgreSQL COPY Command)

```
COPY Books(Book_ID, Title, Author, Genre, Published_Year, Price, Stock) FROM 'D:\30 Day - SQL Practice Files\Books.csv' DELIMITER ',' CSV HEADER; COPY Customers(Customer_ID, Name, Email, Phone, City, Country) FROM 'D:\30 Day - SQL Practice Files\Customers.csv' DELIMITER ',' CSV HEADER; COPY Orders(Order_ID, Customer_ID, Book_ID, Order_Date, Quantity, Total_Amount) FROM 'D:\30 Day - SQL Practice Files\Orders.csv' DELIMITER ',' CSV HEADER;
```

2. Basic SQL Queries (Part 1: Queries 1 - 6)

Q1) Retrieve all books in the "Fiction" genre:

```
SELECT * FROM Books WHERE Genre = 'Fiction';
```

Q2) Find books published after the year 1950:

```
SELECT * FROM Books WHERE Published_Year > 1950;
```

Q3) List all customers from Canada:

```
SELECT * FROM Customers WHERE Country = 'Canada';
```

Q4) Show orders placed in November 2023:

```
SELECT * FROM Orders WHERE Order_Date BETWEEN '2023-11-01' AND '2023-11-30';
```

Q5) Retrieve the total stock of books available:

```
SELECT SUM(Stock) AS Total_Stock FROM Books;
```

Note: Total Stock ke liye `SUM(Stock)` accurate aggregation rule apply hota hai.

Q6) Find the details of the most expensive book:

```
SELECT * FROM Books ORDER BY Price DESC LIMIT 1;
```

3. Basic SQL Queries (Part 2: Queries 7 - 11)

Q7) Show all orders where quantity is greater than 1:

```
SELECT * FROM Orders WHERE Quantity > 1;
```

Q8) Retrieve all orders where the total amount exceeds $20:

```
SELECT * FROM Orders WHERE Total_Amount > 20;
```

Q9) List all distinct genres available in the Books table:

```
SELECT DISTINCT Genre AS Available_Genres FROM Books;
```

Q10) Find the book with the lowest stock:

```
SELECT * FROM Books ORDER BY Stock ASC LIMIT 1;
```

Q11) Calculate the total revenue from all orders:

```
SELECT SUM(Total_Amount) AS Total_Revenue FROM Orders;
```

Note: Entire business ka total revenue `SUM(Total_Amount)` query se calculate hoga.

4. Advanced SQL Queries (Queries 12 - 20)

Q12) Retrieve the number of books sold for each genre:

```
SELECT b.Genre, SUM(o.Quantity) AS Total_Books_Sold FROM Books b JOIN Orders o ON b.Book_ID = o.Book_ID GROUP BY b.Genre;
```

Q13) Find the average price of books in the 'Fantasy' genre:

```
SELECT AVG(Price) AS Avg_Fantasy_Price FROM Books WHERE Genre = 'Fantasy';
```

Q14) List customers who have placed at least 2 (more than 1) orders:

```
SELECT o.Customer_ID, c.Name, COUNT(o.Order_ID) AS Total_Orders FROM Orders o JOIN Customers c ON c.Customer_ID = o.Customer_ID GROUP BY o.Customer_ID, c.Name HAVING COUNT(o.Order_ID) >= 2;
```

Q15) Find the most frequently ordered book:

```
SELECT o.Book_ID, b.Title, COUNT(o.Order_ID) AS Order_Count FROM Orders o JOIN Books b ON b.Book_ID = o.Book_ID GROUP BY o.Book_ID, b.Title ORDER BY Order_Count DESC LIMIT 1;
```

Q16) Show top 3 most expensive books in 'Fantasy' genre:

```
SELECT * FROM Books WHERE Genre = 'Fantasy' ORDER BY Price DESC LIMIT 3;
```

Q17) Total quantity of books sold by each author:

```
SELECT b.Author, SUM(o.Quantity) AS Total_Books_Sold FROM Books b JOIN Orders o ON b.Book_ID = o.Book_ID GROUP BY b.Author ORDER BY Total_Books_Sold DESC;
```

Q18) Cities where customers spent total revenue > $250:

```
SELECT c.City, SUM(o.Total_Amount) AS Total_Spent FROM Orders o JOIN Customers c ON o.Customer_ID = c.Customer_ID GROUP BY c.City HAVING SUM(o.Total_Amount) > 250;
```

Q19) Top spending customers (Highest Total Revenue):

```
SELECT c.Customer_ID, c.Name, SUM(o.Total_Amount) AS Total_Spent FROM Customers c JOIN Orders o ON o.Customer_ID = c.Customer_ID GROUP BY c.Customer_ID, c.Name ORDER BY Total_Spent
```